



Universidad Rey Juan Carlos

Aplicaciones Industriales de Visión Artificial

Manual de Usuario

Conteo automático de glóbulos rojos

Proyecto: AIVA_2026-RBC_Counter

Autores: Daniel Cámara Núñez y Carla Barbero Martín

Fecha: Mayo 2026

Índice

1. Introducción	2
2. Guía del instalador	2
2.1. Introducción	2
2.2. Estructura del sistema	2
2.3. Acceso a la Raspberry Pi	2
2.4. Despliegue estándar	3
2.5. Despliegue mínimo	5
2.6. Despliegue máximo	6
2.7. Despliegue de desarrollo	7
3. Guía del usuario	8
3.1. Flujo general de uso	8
3.2. Casos de prueba	9
3.3. Resultados del sistema de evaluación	9
3.4. Repositorio GitHub	10

1. Introducción

El sistema **RBC Counter** es una aplicación de visión artificial diseñada para automatizar el conteo de glóbulos rojos en imágenes de plasma sanguíneo obtenidas mediante microscopio.

Este producto permite a un usuario sin conocimientos avanzados de visión artificial ejecutar un proceso completo de detección, conteo y generación de resultados estructurados en formato XML, junto con imágenes de salida que muestran las *bounding boxes* obtenidas.

El objetivo principal es reducir el tiempo de análisis manual realizado por técnicos de laboratorio y facilitar la supervisión posterior mediante documentación gráfica y métricas de calidad.

2. Guía del instalador

2.1. Introducción

La instalación estándar del sistema se realiza sobre un entorno Python, idealmente en una Raspberry Pi 5 con Raspberry Pi OS. El manual explica la instalación básica, la ejecución de un caso de prueba y el despliegue recomendado para uso de producción.

2.2. Estructura del sistema

El sistema se organiza en una serie de carpetas que contienen los distintos componentes necesarios para su funcionamiento:

- **data/**: contiene los datos de entrada del sistema.
 - **JPEGImages/**: imágenes de entrada.
 - **Annotations/**: anotaciones en formato XML (ground truth).
- **src/**: contiene el código fuente del sistema, incluyendo los módulos de procesamiento, detección y evaluación.
- **output/**: almacena los resultados generados por el sistema, como imágenes con *bounding boxes* y archivos XML de salida.
- **models/**: contiene los modelos entrenados utilizados por el sistema (en caso de emplear el despliegue máximo).
- Scripts principales:
 - **run.py**: ejecución principal del sistema sobre una imagen.
 - **run_gt.py**: evaluación de resultados frente a *groundtruth* para una imagen específica.
 - **run_gt_all.py**: evaluación de resultados frente a *groundtruth* para todo el dataset.
 - **run_train.py**: entrenamiento del modelo de detección.

2.3. Acceso a la Raspberry Pi

Para instalar y ejecutar el sistema en una Raspberry Pi, es necesario acceder al dispositivo de forma remota o local.

Opción 1: Acceso por SSH

Si la Raspberry Pi está conectada a la red, puede accederse mediante SSH desde otro equipo ya que se ha establecido una ip estática:

```
ssh rbccounter@192.168.1.190
```

El sistema pedirá una contraseña, introducir: 1234

Opción 2: Acceso local

También es posible conectar directamente:

- monitor (micro HDMI)
- teclado y ratón

Una vez dentro del sistema, se pueden seguir los pasos de instalación estándar descritos en la siguiente sección.

2.4. Despliegue estándar

El despliegue estándar consiste en descargar el repositorio, configurar un entorno virtual Python, instalar las dependencias y ejecutar los scripts principales que procesan imágenes y comparan resultados con anotaciones de ground truth. Se trata de la configuración completa necesaria para instalar y utilizar el sistema en un entorno de ejecución.

Pasos de la instalación estándar

1. Clona el repositorio desde GitHub en el directorio de preferencia y accede a la carpeta del proyecto:

```
git clone https://github.com/DaniCamaraN/AIVA_2026-RBC_Counter.git
cd AIVA_2026-RBC_Counter
```

2. Crea y activa un entorno virtual:

- En Windows:

```
python -m venv .venv
.venv\Scripts\activate
```

- En Linux/Mac o Raspberry Pi OS:

```
python3 -m venv .venv
source .venv/bin/activate
```

3. Instala las dependencias: Debido a limitaciones de espacio en algunos dispositivos (como Raspberry Pi), se recomienda instalar ciertas librerías sin dependencias adicionales.

```
pip install -r requirements.txt
```

En caso de que se reciba un error relacionado con el espacio al instalar el archivo requirements deberá eliminar .txt las librerías ultralytics y ultralytics-thop con el comando:

```
nano requirements.txt
```

Una vez eliminadas se debe hacer `ctrl + X` y `Y` para guardar los cambios. Para instalar estas librerías sin errores ejecutar:

```
pip install ultralytics --no-deps
pip install ultralytics-thop --no-deps
```

4. Confirma que las carpetas de datos existen:

- data/JPEGImages
- data/Annotations

5. Genera el conjunto de datos con las particiones de entrenamiento, validación y test:

```
python run_dataset.py
```

6. Verifica la instalación: Para comprobar que la instalación se ha realizado correctamente, se puede ejecutar alguno de los archivos generales del proceso.

```
python run.py
```

Si la ejecución es correcta, el sistema generará resultados en la carpeta `output`. Para verificarlo:

```
ls output
```

Se mostrarán las carpetas correspondientes a las imágenes procesadas, incluyendo los resultados generados (imágenes con detecciones y archivos XML).

7. (Opcional) Evalúa los resultados obtenidos:

```
python run_gt_all.py
```

Este script compara los resultados generados con las anotaciones de ground truth y calcula métricas como precisión, recall, TP, FP y FN, además de devolver gráficas como ROC, Precision-Recall o matriz de confusión.

Diagrama UML del despliegue estándar

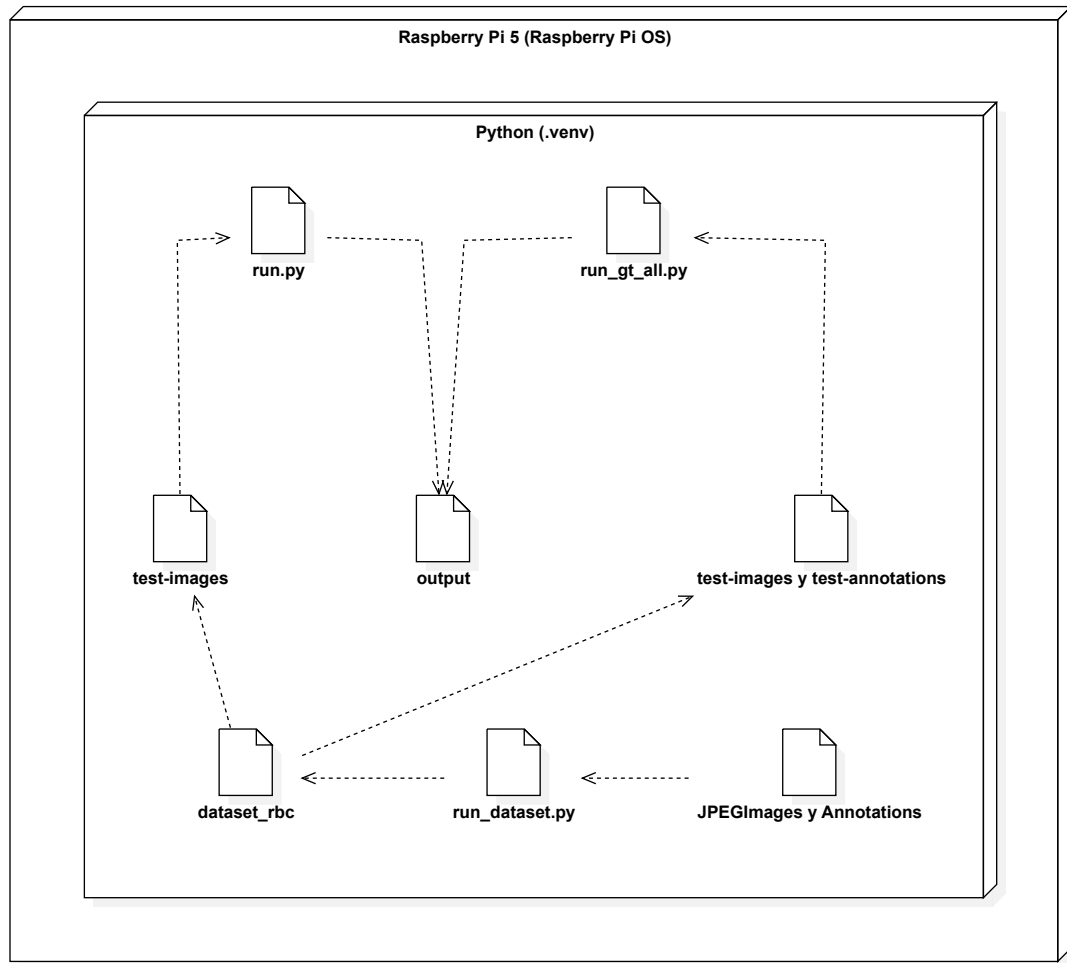


Figura 1: Diagrama UML para el despliegue estandar

2.5. Despliegue mínimo

Este despliegue asume que el sistema ya ha sido instalado previamente. Es una ejecución básica en la que el sistema está completamente instalado y el usuario solo debe ejecutar los archivos proporcionados. Para realizar este tipo de despliegue, el usuario debe ejecutar el archivo de generación de resultados:

```
python run.py
```

De este modo, el sistema procesa una única imagen de ejemplo y genera el resultado en `output/`. En caso de que el usuario quiera verificar las métricas de evaluación deberá ejecutar:

```
python run_gt.py
```

De esta forma, los resultados obtenidos sobre la imagen de test especificada se compararán con el *groundtruth* y devolverá las métricas oportunas.

Diagrama UML del despliegue mínimo

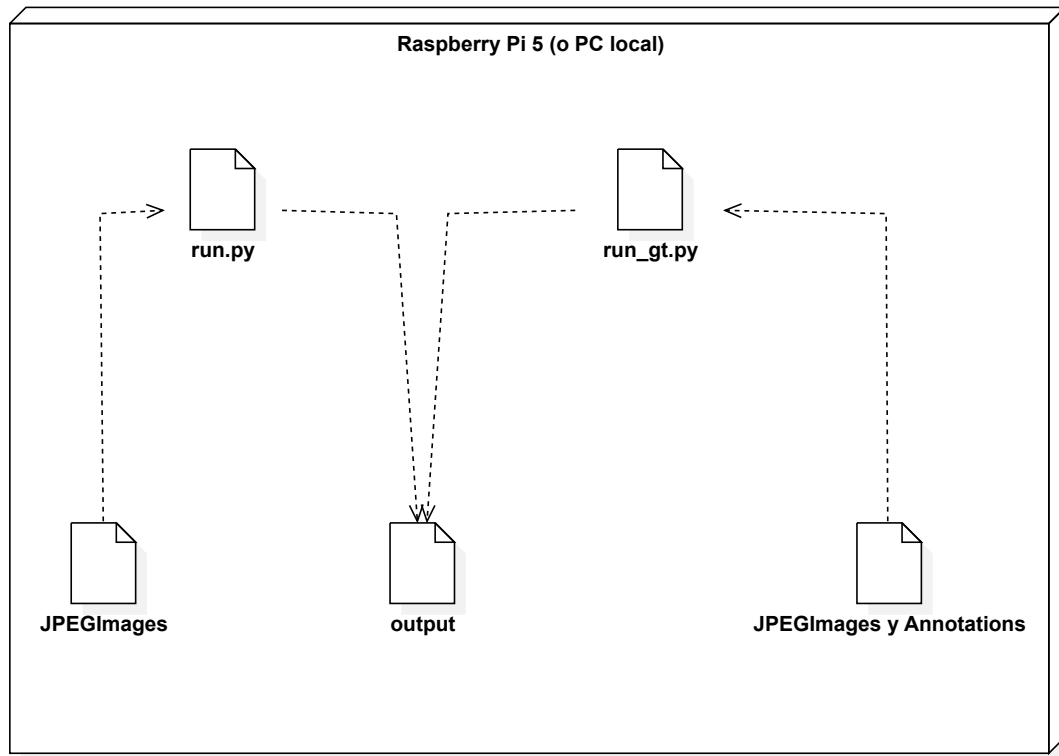


Figura 2: Diagrama UML para el despliegue mínimo

2.6. Despliegue máximo

El despliegue máximo incluye no solo el procesamiento y la evaluación, sino también la preparación del dataset y el entrenamiento del modelo de detección. Este despliegue es adecuado si se desea regenerar el modelo localmente o adaptar el entrenamiento a nuevas anotaciones.

Este despliegue se basaría en una serie de pasos adicionales al despliegue estándar. Antes de ejecutar el archivo `run.py`:

1. Prepara los datos y realiza las particiones de entrenamiento, validación y test:

```
python run_dataset.py
```

2. Ejecuta el entrenamiento del modelo:

```
python run_train.py
```

El entrenamiento genera los archivos de modelo en la carpeta `models/`.

3. Tras el entrenamiento, ejecuta el procesamiento (`run.py`) y la evaluación como en el despliegue estándar (`run_gt_all.py`).

Diagrama UML del despliegue máximo

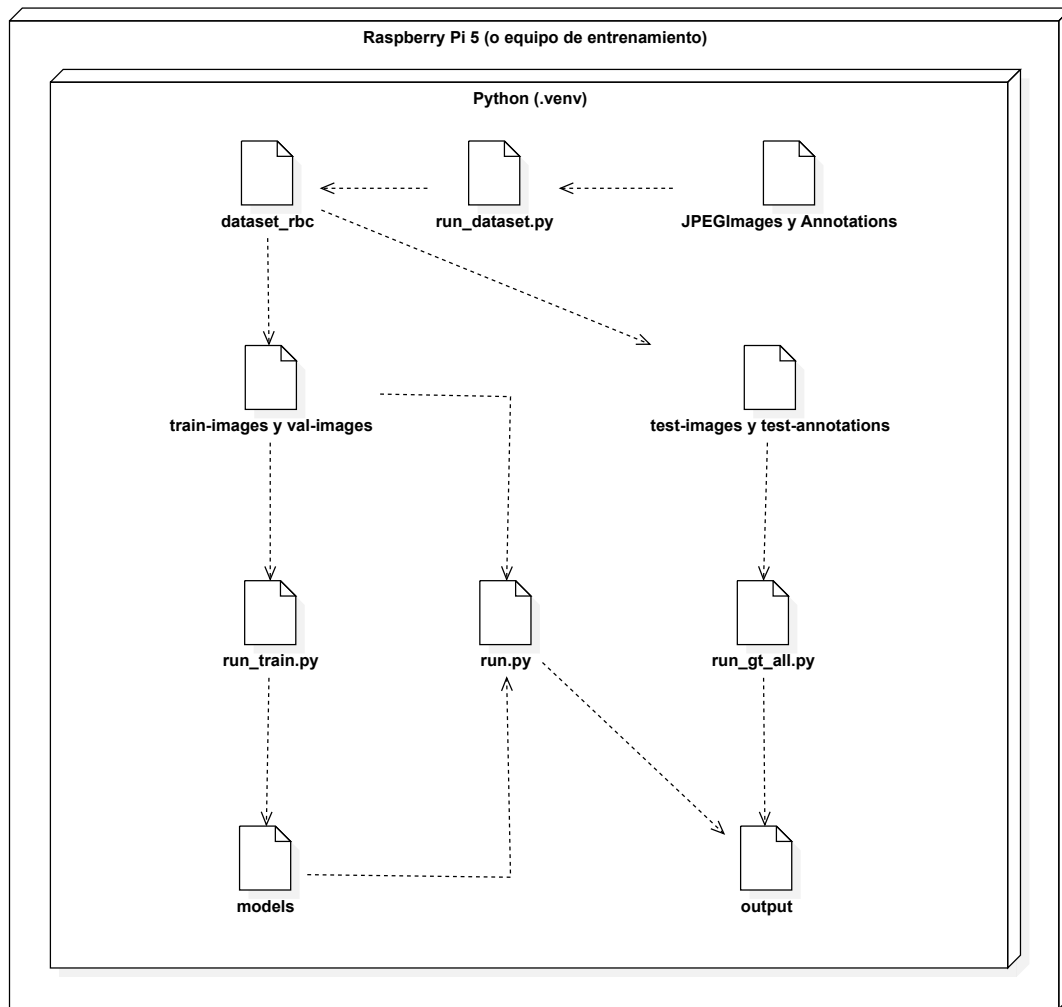


Figura 3: Diagrama UML para el despliegue máximo

2.7. Despliegue de desarrollo

El despliegue de desarrollo permite modificar el código fuente del proyecto en caso de que fuera necesario. Además de los pasos de instalación estándar, el despliegue de desarrollo convella algunos pasos adicionales.

- Abre el proyecto en un editor de código (por ejemplo Visual Studio Code).
- Ejecuta y depura los scripts dentro del entorno virtual.
- Modifica los módulos de `src/` si es necesario.

En este modo se puede probar cualquier cambio usando los scripts:

```
python run.py
python run_gt.py
python run_gt_all.py
python run_train.py
python run_dataset.py
```


Diagrama UML del despliegue de desarrollo

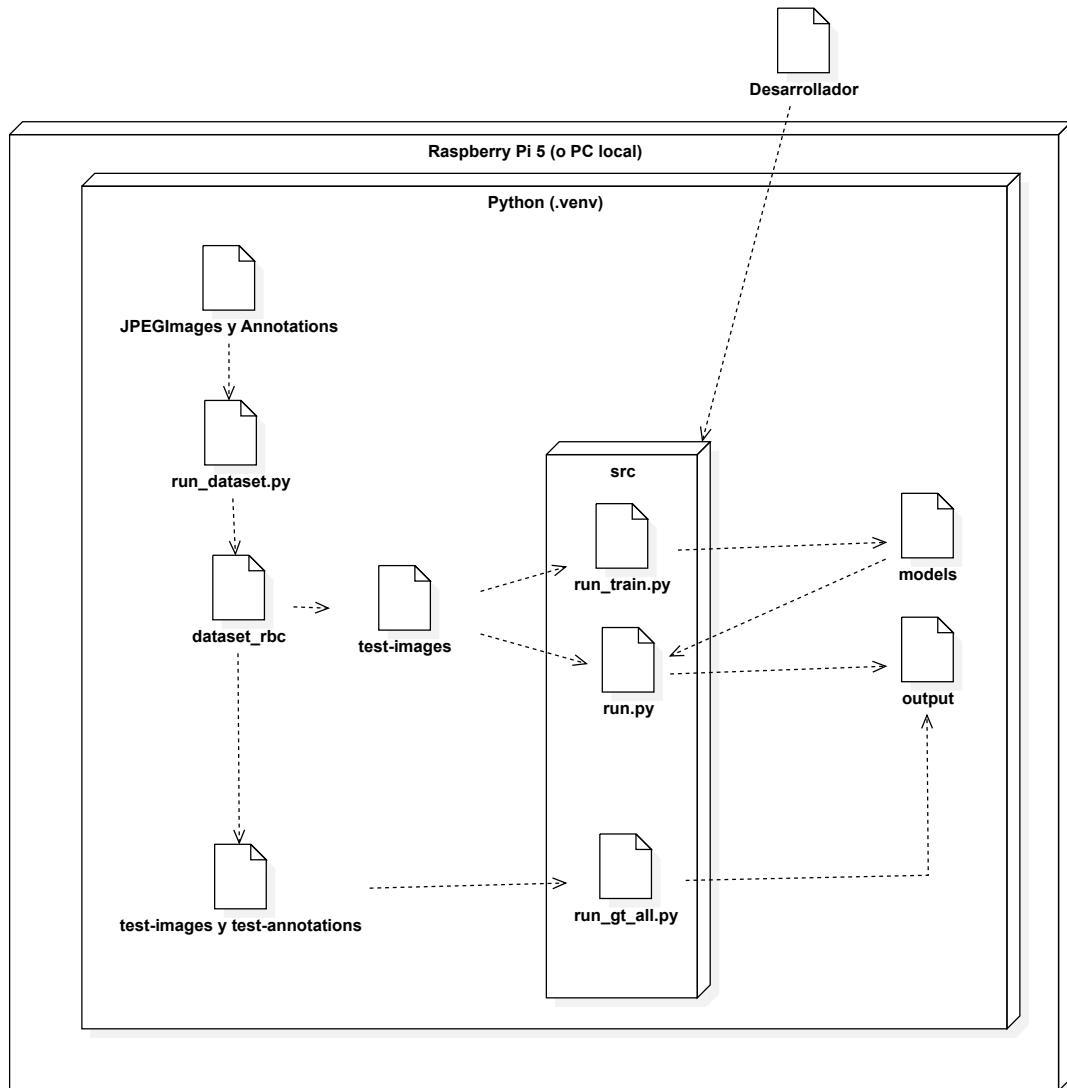


Figura 4: Diagrama UML para el despliegue de desarrollo

3. Guía del usuario

En esta sección se explica, paso a paso, cómo usar el sistema para procesar un caso de ejemplo.

3.1. Flujo general de uso

El uso típico del sistema sigue los siguientes pasos:

1. Preparar las imágenes y anotaciones en las carpetas `data/JPEGImages` y `data/Annotations`.
2. Ejecutar el procesamiento de las imágenes.
3. Revisar los resultados generados en la carpeta `output/`.
4. (Opcional) Evaluar los resultados mediante métricas.

3.2. Casos de prueba

Procesar una sola imagen

1. Asegurarse de que el entorno virtual está activo y las dependencias instaladas.
2. Comprobar que la imagen de prueba existe (`data/JPEGImages/BloodImage_00000.jpg`).
3. (Opcional para evaluación) Comprobar que el XML de ground truth existe (`data/Annotations/BloodImage_00000.xml`).
4. Ejecutar el procesamiento:

```
python run.py
```

5. (Opcional) Evaluar los resultados:

```
python run_gt.py
```

6. Revisar los resultados en la carpeta `output/`.

En la carpeta `output/`, el usuario encontrará:

- Imágenes con las detecciones realizadas (*bounding boxes*).
- Archivos XML con los resultados estructurados.

Procesar todo el dataset

Para procesar todas las imágenes disponibles y generar métricas por imagen, se debe usar el script:

```
python run_gt_all.py
```

Este comando:

- procesa todas las imágenes de `data/JPEGImages`;
- compara cada imagen con su XML correspondiente en `data/Annotations`;
- registra el tiempo de procesado por imagen;
- muestra el tiempo total de ejecución al final.

3.3. Resultados del sistema de evaluación

Al finalizar la ejecución, el sistema muestra:

- el número de imágenes procesadas y fallidas
- métricas promedio de precisión y recall
- tiempos de procesamiento (total, medio, mínimo y máximo)
- métricas acumuladas de TP, FP y FN

Los resultados generados también quedarán almacenados en la carpeta `output/` como archivos XML y como imágenes con las *bounding boxes* superpuestas.

3.4. Repositorio GitHub

Toda la información del proyecto y el código fuente se alojan en GitHub:

https://github.com/DaniCamaraN/AIVA_2026-RBC_Counter

El repositorio contiene el código de la aplicación, los scripts de prueba y las instrucciones necesarias para ejecutar el sistema.